Всероссийская олимпиада студентов «Я – профессионал»

Направление: «Программная инженерия»

Задание № 1. Оптимальная отгрузка

Проектное решение автоматизированной системы подбора транспорта для отгрузки труб

Категория участия: бакалавриат

1. Введение

Цель работы — спроектировать автоматизированную web-систему, которая по заданному заказу на трубы определяет минимально необходимое количество единиц транспорта для отгрузки. Система должна учитывать сортамент труб, количество, длину, справочные ограничения по размещению в транспорте, а также выбранный пользователем приоритетный тип перевозки: железнодорожный вагон или автотранспорт.

Предметная область содержит два класса данных. Первый класс — относительно стабильные справочники: сортамент труб, типы транспорта, нормативная вместимость. Второй класс — операционные данные: заказы, позиции заказов, результаты расчета, действия пользователей. Это позволяет сразу выбрать реляционную модель хранения и разделить сущности таким образом, чтобы убрать дублирование и обеспечить третью нормальную форму.

Основная функция системы формализуется как задача детерминированного подбора транспорта по справочным ограничениям. Для каждого сортамента и выбранного типа транспорта известна допустимая вместимость. Дальше вычисляется минимальное число единиц транспорта по формуле ceil(quantity / capacity). Такой подход лучше подходит для производственного учета, чем общий 3D bin packing: решение прозрачно, воспроизводимо и легко проверяется.

1.1. Использование ИИ-инструментов

При подготовке пояснительной записки использовался DeepSeek как вспомогательный инструмент для структурирования материала, проверки полноты разделов и черновой редакции формулировок. DeepSeek не использовался как средство автоматического получения готового итогового решения без анализа. Постановка задачи, декомпозиция предметной области, выбор сущностей, проектирование базы данных, архитектурное решение,

алгоритм, тестовые сценарии, программная реализация и финальная редакция содержания выполнены автором самостоятельно.

Ниже приведены примеры точечных промптов, использованных как вспомогательные:

- Предложи набор сущностей реляционной БД для системы расчета отгрузки труб с учетом 3НФ и разделения справочников и операционных данных.
- Сравни монолитную трехслойную архитектуру и микросервисный вариант для учебного web-прототипа системы подбора транспорта.
- Помоги формализовать алгоритм расчета минимального количества транспорта по справочной вместимости и ограничениям выбранного типа перевозки.
- Составь перечень сценариев ошибочного ввода для системы расчета отгрузки труб.
- Предложи структуру DTO и сервисного слоя для backend-части системы.

1.2. Принятые допущения по исходным данным

В приложенном условии для части сортаментов вместимость приведена диапазонами. Для демонстрационной программной реализации диапазоны нормализованы к консервативной нижней границе. Это гарантирует, что рассчитанный план отгрузки не нарушает табличные ограничения.

Вес в примерах из условия не позволяет однозначно восстановить единую формулу для всех строк, поэтому в проекте нормативная вместимость вынесена в отдельный справочник TransportPipeCapacity. В промышленном варианте эта таблица заполняется технологом или логистом на основании подтвержденных расчетов по геометрии и грузоподъемности.

1.3. Карта соответствия критериям оценивания

Чтобы упростить проверку работы, ниже приведено сопоставление обязательных пунктов задания с разделами пояснительной записки и приложениями.

Критерий из задания	Где раскрыт в работе
Введение и указание ИИ-инструмента	Раздел 1, подпункты 1.1–1.2
2.1. Инфологическая и даталогическая модель БД	Разделы 2.1–2.2, рисунок 1, таблицы структуры БД
2.2. Архитектура решения, классы, DTO, паттерны	Раздел 2.3, рисунок 2
2.3.1. Блок-схема алгоритма	Раздел 2.5, рисунок 3
2.3.2. Программная реализация алгоритма	Раздел 2.5 и приложение А, файл transport_optimizer.py
2.3.3. Проверка на корректных и ошибочных примерах	Раздел 2.6, таблица контрольных сценариев
2.4–2.5. Технологии и прототипы интерфейсов	Разделы 2.4 и 2.7, рисунки 4–5

2. Основная часть

2.1. Инфологическая модель предметной области

В инфологической модели выделены сущности, которые реально существуют в предметной области и используются системой: пользователь, роль, заказ, позиция заказа, сортамент трубы, тип транспорта, нормативная вместимость, результат расчета и единица результата расчета. Схема строится так, чтобы

справочники не смешивались с операционными документами, а результат расчета мог быть сохранен и воспроизведен.

Рисунок 1 — Инфологическая ER-модель предметной области

Ключевые связи: Order -> OrderItem (1:M), PipeStandard -> OrderItem (1:M), TransportType -> TransportPipeCapacity (1:M), PipeStandard -> TransportPipeCapacity (1:M), CalculationResult -> CalculationResultItem (1:M), CustomerOrder -> CalculationResult (1:M), Role -> User (1:M).

Обоснование 3НФ: атрибуты каждой таблицы зависят только от первичного ключа; транзитивные зависимости вынесены в отдельные сущности; справочная вместимость не хранится в заказе, а определяется через связку "тип транспорта + сортамент трубы"; пользователи и роли отделены от операционных данных.

2.2. Даталогическая модель базы данных

Физическая модель БД ориентирована на PostgreSQL. Ниже приведены таблицы, поля, типы данных и назначение ключевых атрибутов.

Таблица Role

Поле	Тип данных	Ключ	Назначение
role_id	BIGSERIAL	PK	Идентификатор роли
name	VARCHAR(50)	UNIQUE	Наименование роли

Таблица User

Поле	Тип данных	Ключ	Назначение
user_id	BIGSERIAL	PK	Идентификатор пользователя
login	VARCHAR(100)	UNIQUE	Логин
password_hash	VARCHAR(255)	-	Хеш пароля
role_id	BIGINT	FK -> Role.role_id	Роль пользователя
is_active	BOOLEAN	-	Признак активности

Таблица PipeStandard

Поле	Тип данных	Ключ	Назначение
pipe_standard_id	BIGSERIAL	PK	Идентификатор сортамента
diameter_mm	INTEGER	-	Наружный диаметр
wall_thickness_mm	INTEGER	-	Толщина стенки
length_m	NUMERIC(5,2)	-	Длина трубы

Поле	Тип данных	Ключ	Назначение
weight_per_pipe_t	NUMERIC(8,3)	-	Масса одной трубы

Таблица TransportType

Поле	Тип данных	Ключ	Назначение
transport_type_id	BIGSERIAL	PK	Идентификатор типа транспорта
name	VARCHAR(120)	-	Наименование
transport_category	VARCHAR(20)	-	WAGON или TRUCK
payload_t	NUMERIC(8,2)	-	Грузоподъемность
inner_width_m	NUMERIC(5,2)	-	Полезная ширина
inner_height_m	NUMERIC(5,2)	-	Полезная высота
inner_length_m	NUMERIC(5,2)	-	Полезная длина
is_active	BOOLEAN	-	Доступность в расчете

Таблица TransportPipeCapacity

Поле	Тип данных	Ключ	Назначение
capacity_id	BIGSERIAL	PK	Идентификатор записи вместимости
transport_type_id	BIGINT	FK -> TransportType.transport_type_id	Тип транспорта

Поле	Тип данных	Ключ	Назначение
pipe_standard_id	BIGINT	FK -> PipeStandard.pipe_standard_id	Сортамент трубы
max_pipes_allowed	INTEGER	-	Нормативная вместимость
source_note	VARCHAR(255)	-	Источник нормы/комментарий

Таблица CustomerOrder

Поле	Тип данных	Ключ	Назначение
order_id	BIGSERIAL	PK	Идентификатор заказа
created_at	TIMESTAMP	-	Дата создания
priority_transport_category	VARCHAR(20)	-	Приоритетный тип транспорта
status	VARCHAR(30)	-	Статус заказа

Таблица CustomerOrderItem

Поле	Тип данных	Ключ	Назначение
order_item_id	BIGSERIAL	PK	Идентификатор позиции заказа
order_id	BIGINT	FK -> CustomerOrder.order_id	Ссылка на заказ

Поле	Тип данных	Ключ	Назначение
pipe_standard_id	BIGINT	FK -> PipeStandard.pipe_standard_id	Сортамент
quantity	INTEGER	-	Количество труб

Таблица CalculationResult

Поле	Тип данных	Ключ	Назначение
result_id	BIGSERIAL	PK	Идентификатор расчета
order_id	BIGINT	FK -> CustomerOrder.order_id	Заказ
calculated_at	TIMESTAMP	-	Время расчета
total_units	INTEGER	-	Общее количество единиц транспорта

Таблица CalculationResultItem

Поле	Тип данных	Ключ	Назначение
result_item_id	BIGSERIAL	PK	Идентификатор строки результата
result_id	BIGINT	FK -> CalculationResult.result_id	Ссылка на расчет

Поле	Тип данных	Ключ	Назначение
transport_type_id	BIGINT	FK -> TransportType.transport_type_id	Тип транспорта
pipe_standard_id	BIGINT	FK -> PipeStandard.pipe_standard_id	Сортамент трубы
pipes_loaded	INTEGER	-	Сколько труб загружено
unit_index	INTEGER	-	Порядковый номер единицы транспорта

Таблица AuditLog

Поле	Тип данных	Ключ	Назначение
audit_id	BIGSERIAL	PK	Идентификатор события
user_id	BIGINT	FK -> User.user_id	Пользователь
entity_name	VARCHAR(80)	-	Измененная сущность
entity_id	BIGINT	-	Идентификатор записи
action_type	VARCHAR(30)	-	INSERT/UPDATE/DELETE
changed_at	TIMESTAMP	-	Время изменения

Ограничения целостности: для PipeStandard вводится уникальный индекс по набору (diameter_mm, wall_thickness_mm, length_m); для TransportPipeCapacity — уникальный индекс по паре (transport_type_id, pipe_standard_id); для quantity и max_pipes_allowed задаются CHECK > 0.

Такой набор ограничений упрощает валидацию и предотвращает появление конфликтующих записей.

2.3. Архитектура решения

Для системы выбрана трехслойная web-архитектура. Она достаточно проста для учебного прототипа, но при этом закрывает все требования задания: отдельные пользовательские интерфейсы, изолированный слой бизнес-логики, централизованную работу со справочниками и возможность разграничения прав доступа.

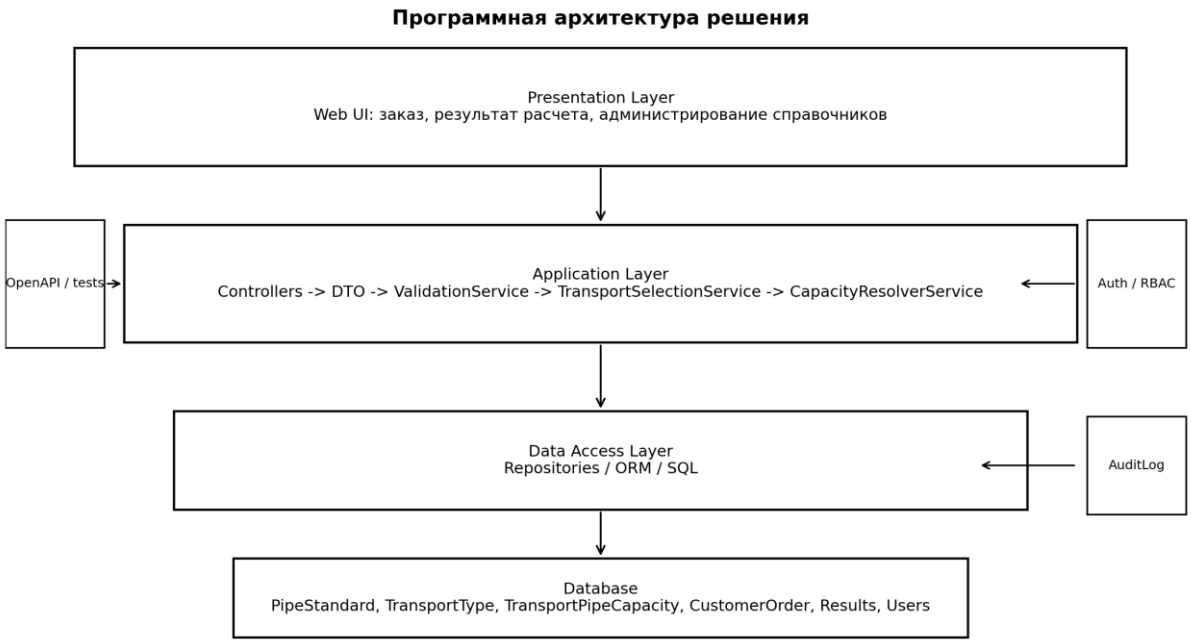

Рисунок 2 — Архитектурная схема решения

Состав слоев:

- Presentation Layer — страницы ввода заказа, вывода результата и административного редактирования справочников.
- Application Layer — контроллеры, DTO, валидация, сервис расчета транспорта, сервис определения нормативной вместимости.
- Data Access Layer — репозитории и ORM-модели.

- Database Layer — PostgreSQL с ограничениями целостности и журналом изменений.

Основные классы и сервисы:

- OrderController — прием заказа и запуск расчета.
- ReferenceDataController — CRUD-операции по сортаменту, транспорту и справочной вместимости.
- TransportSelectionService — основной алгоритм подбора транспорта.
- CapacityResolverService — получение нормативной вместимости по паре "транспорт + сортамент".
- ValidationService — проверка корректности пользовательского ввода.
- OrderRepository / PipeStandardRepository / TransportTypeRepository / CalculationResultRepository — слой доступа к данным.

Используемые DTO:

- CreateOrderRequestDTO: priorityTransport, items[].
- OrderItemDTO: diameterMm, wallThicknessMm, lengthM, weightPerPipeT, quantity.
- CalculationResponseDTO: totalUnits, resultItems, warnings.
- TransportUnitDTO: transportName, transportCategory, unitIndex, pipesLoaded.
- ReferencePipeDTO / ReferenceTransportDTO — обмен данными со справочниками.

Архитектурные паттерны: Layered Architecture, Repository, Service, DTO. Отдельно поддерживается RBAC-модель ролей: рядовой пользователь может создавать заказы и запускать расчет; редактирование справочников доступно только пользователю с административной ролью.

2.4. Технологии, методы и средства разработки

Backend: Python 3.11 + FastAPI. Выбор обусловлен высокой скоростью реализации, удобной валидацией входных данных через Pydantic и автоматическим формированием REST-документации.

База данных: PostgreSQL. Это стандартная промышленная реляционная СУБД с поддержкой внешних ключей, ограничений CHECK, индексов и журналирования изменений.

ORM: SQLAlchemy. Она позволяет отделить бизнес-логику от SQL-кода и упростить поддержку модели данных.

Frontend-прототип: HTML/CSS/JavaScript либо Figma-макеты. Для задания достаточно низко- и среднедетализированных прототипов, которые показывают структуру экранов и пользовательские сценарии.

Средства моделирования: draw.io / PlantUML / Mermaid. Для итоговой записки допускается экспорт диаграмм в растровом виде при сохранении читаемости.

Метод разработки: сначала фиксируется модель данных, затем слой бизнес-логики, затем пользовательские сценарии. Такой порядок минимизирует переделки, потому что основной риск задачи находится в некорректной формализации предметной области, а не во внешнем интерфейсе.

2.5. Основной алгоритм системы

Алгоритм решает задачу поиска минимального количества единиц транспорта для каждой позиции заказа. Он не пытается строить сложную геометрическую упаковку "с нуля", а работает по нормативной вместимости, вынесенной в отдельный справочник. Это соответствует практической постановке: в реальной логистике норматив размещения по сортаменту чаще всего заранее утвержден.

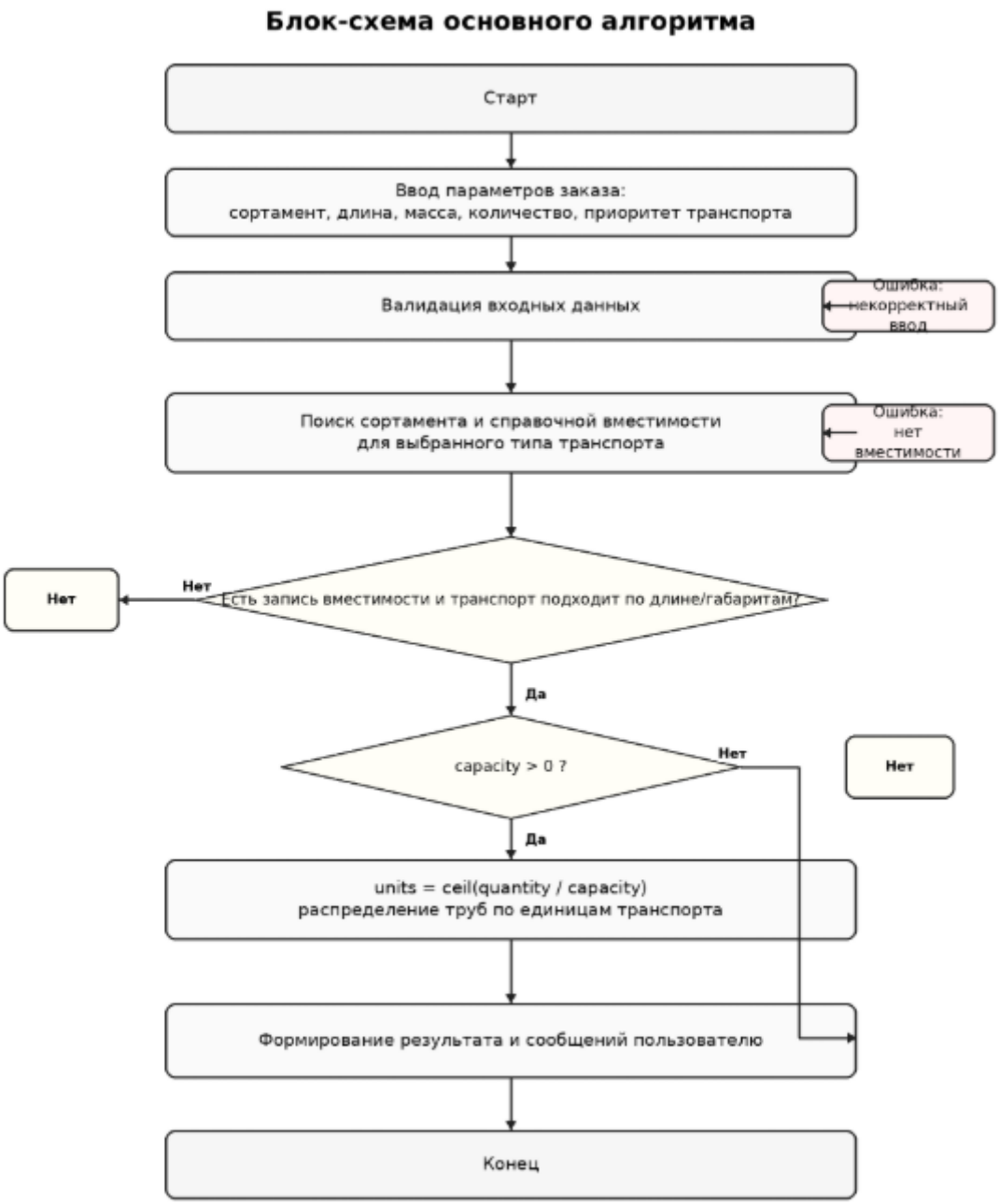

Рисунок 3 — Блок-схема основного алгоритма

Входные данные: для каждой позиции заказа передаются диаметр трубы, толщина стенки, длина, масса одной трубы, количество труб и приоритетный тип транспорта.

Выходные данные: список единиц транспорта с указанием типа транспорта, номера единицы, сортамента и количества труб в каждой единице, а также предупреждения и сообщения о некорректных данных.

Пошаговое описание алгоритма:

1. Проверить корректность входных данных: все числовые параметры должны быть заданы и быть положительными.
2. Проверить, помещается ли труба по длине в полезный объем выбранного транспорта.
3. Определить сортамент трубы по ключу (diameter_mm, wall_thickness_mm).
4. Получить нормативную вместимость из справочника TransportPipeCapacity.
5. При наличии точной массы одной трубы дополнительно ограничить вместимость по грузоподъемности: floor(payload / weight_per_pipe).
6. Вычислить итоговую вместимость как минимум из нормативного и весового ограничения.
7. Посчитать число единиц транспорта: units = ceil(quantity / effective_capacity).
8. Сформировать детальный список загрузки по единицам транспорта.
9. Если для выбранного типа транспорта сортамент не поддерживается, вывести ошибку либо использовать резервный тип транспорта в режиме fallback.

Асимптотическая сложность расчета для n позиций заказа — O(n), если справочная вместимость хранится в хеш-таблице или индексированной таблице БД. Память — O(k), где k равно числу строк результата.

Ниже приведен фрагмент программной реализации на Python, реализующий проверки и вычисление минимального количества транспорта:

```python
def optimize_transport(items, priority_transport, allow_fallback=True):
    for item in items:
        validate_item(item)
        selected_transport = priority_transport
        try:
            capacity = get_capacity(item, selected_transport)
        except ValidationError as primary_error:
            if not allow_fallback:
                raise
            capacity = get_capacity(item, fallback)
            selected_transport = fallback

        unit_count = ceil(item.quantity / capacity)
        remaining = item.quantity
        for unit_index in range(1, unit_count + 1):
            loaded = min(capacity, remaining)
            result.append((selected_transport, unit_index, loaded))
            remaining -= loaded
```

2.6. Проверка на примерах входных данных

Для демонстрации были проверены как корректные сценарии, так и сценарии с ошибочным вводом. В тестовых примерах использованы нормативные вместимости из таблиц задания.

Сценарий	Вход	Ожидаемый расчет	Результат системы
Корректный ввод	426x8, длина 11 м, масса 0.95 т, количество 70, приоритет truck	capacity = 19, units = ceil(70 / 19) = 4	Еврофура #1-#3 по 19 труб, Еврофура #4 — 13 труб

Сценарий	Вход	Ожидаемый расчет	Результат системы
Корректный ввод	1020x12, длина 11 м, масса 4.0 т, количество 9, приоритет truck	capacity = 4, units = ceil(9 / 4) = 3	Еврофура #1 — 4, #2 — 4, #3 — 1 труба
Ошибка ввода	426x8, длина 11 м, масса 0.95 т, количество 0	quantity <= 0	Сообщение: "Количество труб должно быть положительным"
Ошибка ввода	530x8, длина 14.5 м, масса 1.10 т, количество 10, приоритет wagon	length > inner_length_m	Сообщение: "Труба длиной 14.5 м не помещается в полувагон 12.118 м"
Человеческий фактор	Пустое поле диаметра или текст вместо числа	Ошибка преобразования / отсутствие обязательного поля	Сообщение о некорректном формате данных

Контрольные сценарии показывают, что система не только строит план отгрузки на корректных данных, но и явно останавливает расчет в ситуациях, где пользователь ввел некорректный объем, отсутствующий сортамент или неподходящую длину трубы. Это соответствует требованию задания о демонстрации работы при человеческом факторе.

2.7. Прототипы визуальных интерфейсов

В проекте предусмотрены две подсистемы интерфейсов: рабочее место пользователя для расчета заказа и административная подсистема для редактирования справочной информации. Прототипы сделаны в структурном виде; их задача — показать логику экранов, а не финальный дизайн.

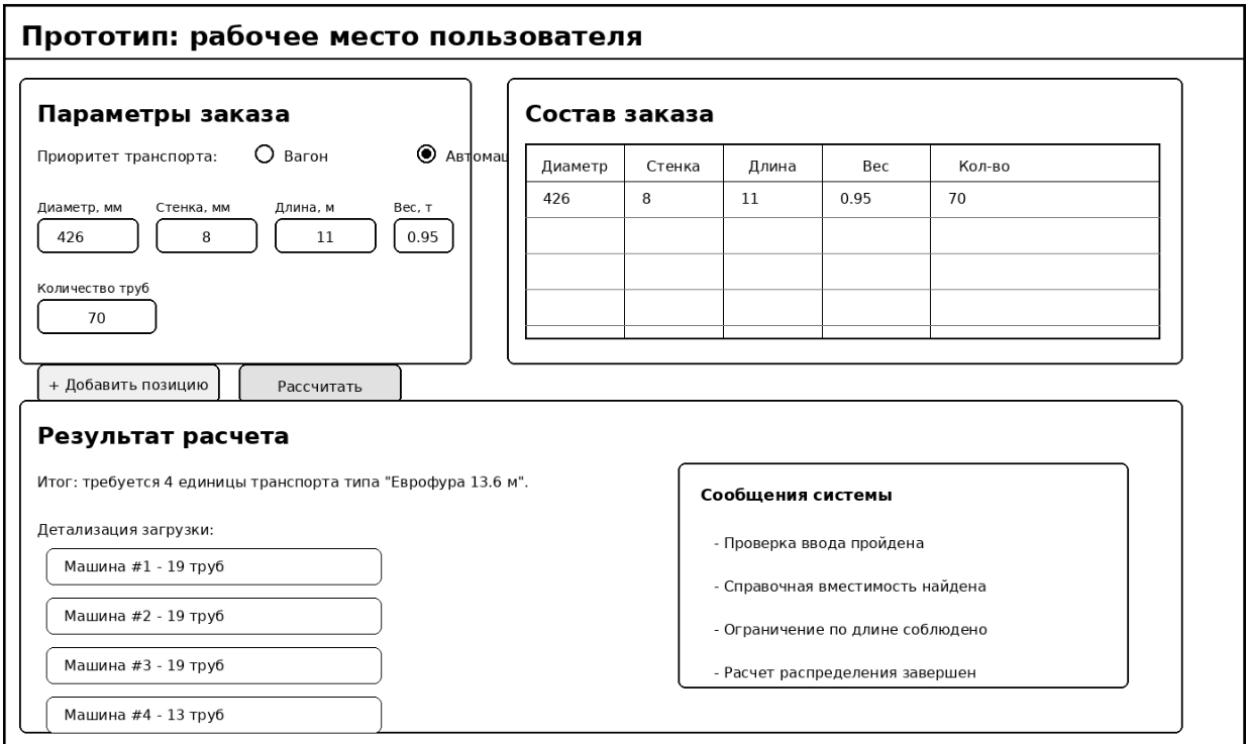

Рисунок 4 — Прототип интерфейса пользователя

На экране пользователя должны быть доступны: выбор приоритетного типа транспорта, ввод параметров каждой позиции заказа, просмотр состава заказа, запуск расчета и получение детализированного результата по каждой единице транспорта.

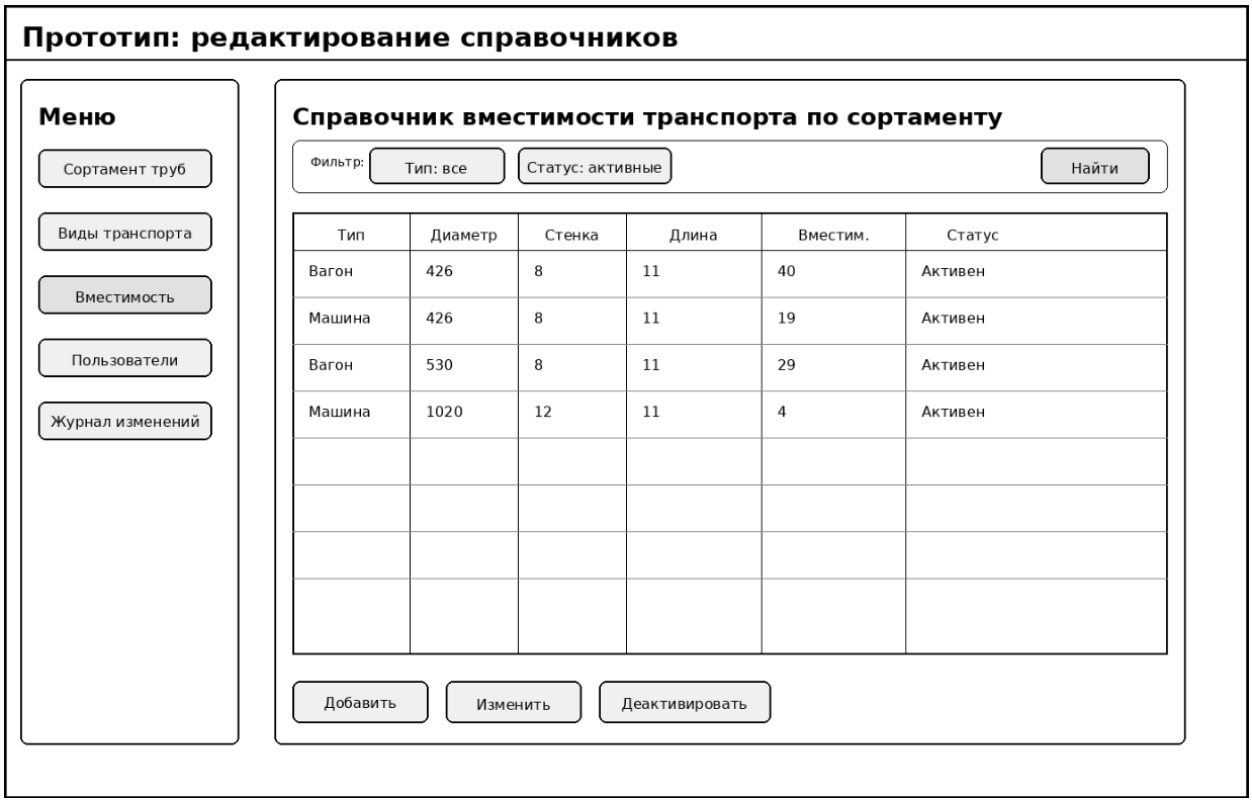

Рисунок 5 — Прототип административного интерфейса

Административный интерфейс включает отдельные справочники сортамента труб, типов транспорта, нормативной вместимости и пользователей. Для справочников предусмотрены операции добавления, изменения, деактивации записей и журналирования действий пользователя.

3. Заключение

В работе спроектирована автоматизированная система подбора транспорта для отгрузки труб. Предметная область формализована через реляционную модель данных, удовлетворяющую третьей нормальной форме. Выделены справочники, операционные сущности и результат расчета. Все обязательные элементы задания представлены в явном виде: модели данных, архитектура, алгоритм, программная реализация, проверка на сценариях и прототипы интерфейсов.

Предложена трехслойная архитектура web-системы с разграничением прав доступа. Основной алгоритм сведен к детерминированному расчету минимального количества транспорта по нормативной вместимости и

ограничениям выбранного типа перевозки. Такой подход прозрачен, легко проверяется и подходит для промышленного внедрения.

Подготовлены прототипы двух подсистем интерфейса, а также программная реализация базового алгоритма с проверкой корректных и ошибочных сценариев ввода. В разделе 1 отдельно зафиксировано использование DeepSeek как вспомогательного инструмента и приведены примеры промптов. В результате получено решение, которое закрывает все пункты задания и может служить основой для дальнейшей разработки полноценного сервиса.

Приложение А. Полный листинг программной реализации

Ниже приведен полный листинг демонстрационной реализации алгоритма на Python. В варианте защиты его можно вынести в отдельный файл и приложить к архиву решения.

```
from __future__ import annotations
from dataclasses import dataclass
from math import ceil
from typing import Dict, List, Tuple


class ValidationError(Exception):
    pass


@dataclass(frozen=True)
class PipeKey:
    diameter_mm: int
    wall_thickness_mm: int


@dataclass
class OrderItem:
    diameter_mm: int
    wall_thickness_mm: int
    length_m: float
    weight_per_pipe_t: float
    quantity: int


@dataclass
```

```
class TransportLoad:
    transport_name: str
    transport_category: str
    pipe_key: PipeKey
    unit_index: int
    pipes_loaded: int


@dataclass
class CalculationResult:
    total_units: int
    loads: List[TransportLoad]
    warnings: List[str]


TRANSPORTS = {
    "wagon": {
        "name": "Полувагон 12.118 м",
        "payload_t": 69.0,
        "inner_length_m": 12.118,
        "inner_width_m": 2.878,
        "inner_height_m": 2.060,
    },
    "truck": {
        "name": "Еврофура 13.6 м",
        "payload_t": 40.0,
        "inner_length_m": 13.6,
        "inner_width_m": 2.45,
        "inner_height_m": 2.40,
    },
```

```
}

# Консервативная вместимость по таблицам задания:
# для диапазонов используется нижняя граница, чтобы результат
гарантированно оставался допустимым.
CAPACITY_TABLE: Dict[str, Dict[PipeKey, int]] = {
    "wagon": {
        PipeKey(273, 6): 103,
        PipeKey(273, 8): 103,
        PipeKey(325, 6): 70,
        PipeKey(325, 7): 70,
        PipeKey(325, 10): 44,
        PipeKey(377, 9): 52,
        PipeKey(426, 7): 40,
        PipeKey(426, 8): 40,
        PipeKey(426, 10): 40,
        PipeKey(530, 8): 29,
        PipeKey(630, 8): 21,
        PipeKey(630, 9): 21,
        PipeKey(720, 8): 13,
        PipeKey(820, 8): 11,
        PipeKey(820, 9): 11,
        PipeKey(920, 8): 16,
        PipeKey(1020, 10): 6,
        PipeKey(1020, 12): 6,
        PipeKey(1020, 14): 5,
    },
    "truck": {
        PipeKey(426, 7): 19,
        PipeKey(426, 8): 19,
```

```
        PipeKey(426, 10): 19,
        PipeKey(530, 8): 16,
        PipeKey(630, 8): 11,
        PipeKey(630, 9): 11,
        PipeKey(720, 8): 8,
        PipeKey(820, 8): 8,
        PipeKey(820, 9): 8,
        PipeKey(920, 8): 5,
        PipeKey(1020, 10): 4,
        PipeKey(1020, 12): 4,
        PipeKey(1420, 14): 2,
    },
}


def validate_item(item: OrderItem) -> None:
    if item.quantity <= 0:
        raise ValidationError("Количество труб должно быть положительным.")
    if item.length_m <= 0:
        raise ValidationError("Длина трубы должна быть положительной.")
    if item.weight_per_pipe_t <= 0:
        raise ValidationError("Масса одной трубы должна быть положительной.")
    if item.diameter_mm <= 0 or item.wall_thickness_mm <= 0:
        raise ValidationError("Диаметр и толщина стенки должны быть
положительными.")


def get_capacity(item: OrderItem, transport_category: str) -> int:
    transport = TRANSPORTS[transport_category]
    if item.length_m > transport["inner_length_m"]:
```

```
        raise ValidationError(
            f"Труба длиной {item.length_m} м не помещается в {transport['name']}
"
            f"по длине полезного объема."
        )

    pipe_key = PipeKey(item.diameter_mm, item.wall_thickness_mm)
    if pipe_key not in CAPACITY_TABLE[transport_category]:
        raise ValidationError(
            f"Для сортамента {item.diameter_mm}x{item.wall_thickness_mm} "
            f"нет справочной вместимости для транспорта '{transport['name']}'."
        )

    table_capacity = CAPACITY_TABLE[transport_category][pipe_key]
    weight_capacity = int(transport["payload_t"] // item.weight_per_pipe_t)
    effective_capacity = min(table_capacity, weight_capacity)

    if effective_capacity <= 0:
        raise ValidationError(
            "По ограничениям грузоподъемности нельзя разместить даже одну
трубу."
        )

    return effective_capacity


def optimize_transport(
        items: List[OrderItem],
        priority_transport: str,
        allow_fallback: bool = True,
```

```
) -> CalculationResult:
    if priority_transport not in TRANSPORTS:
        raise ValidationError("Приоритет транспорта должен быть 'wagon' или
'truck'.")

    fallback = "truck" if priority_transport == "wagon" else "wagon"
    loads: List[TransportLoad] = []
    warnings: List[str] = []

    for item in items:
        validate_item(item)

        selected_transport = priority_transport
        try:
            capacity = get_capacity(item, selected_transport)
        except ValidationError as primary_error:
            if not allow_fallback:
                raise
            try:
                capacity = get_capacity(item, fallback)
                selected_transport = fallback
                warnings.append(
                    f"Для сортамента {item.diameter_mm}x{item.wall_thickness_mm} "
                    f"использован резервный тип транспорта
'{TRANSPORTS[fallback]['name']}', "
                    f"так как приоритетный вариант недоступен."
                )
            except ValidationError:
                raise primary_error
```

```
        unit_count = ceil(item.quantity / capacity)
        remaining = item.quantity
        for unit_index in range(1, unit_count + 1):
            loaded = min(capacity, remaining)
            loads.append(
                TransportLoad(
                    transport_name=TRANSPORTS[selected_transport]["name"],
                    transport_category=selected_transport,
                    pipe_key=PipeKey(item.diameter_mm, item.wall_thickness_mm),
                    unit_index=unit_index,
                    pipes_loaded=loaded,
                )
            )
            remaining -= loaded

    return CalculationResult(total_units=len(loads), loads=loads,
warnings=warnings)


if __name__ == "__main__":
    demo_items = [
        OrderItem(426, 8, 11.0, 0.95, 70),
        OrderItem(1020, 12, 11.0, 4.0, 9),
    ]
    result = optimize_transport(demo_items, priority_transport="truck",
allow_fallback=True)
    print("Всего единиц транспорта:", result.total_units)
    for load in result.loads:
        print(
            f"{load.transport_name} |
```

```
{load.pipe_key.diameter_mm}x{load.pipe_key.wall_thickness_mm} | "
            f"ед. #{load.unit_index} | труб: {load.pipes_loaded}"
        )
    if result.warnings:
        print("Предупреждения:")
        for warning in result.warnings:
            print("-", warning)
```

Результат выполнения

```
Всего единиц транспорта: 7
Еврофура 13.6 м | 426x8 | ед. #1 | труб: 19
Еврофура 13.6 м | 426x8 | ед. #2 | труб: 19
Еврофура 13.6 м | 426x8 | ед. #3 | труб: 19
Еврофура 13.6 м | 426x8 | ед. #4 | труб: 13
Еврофура 13.6 м | 1020x12 | ед. #1 | труб: 4
Еврофура 13.6 м | 1020x12 | ед. #2 | труб: 4
Еврофура 13.6 м | 1020x12 | ед. #3 | труб: 1
```